

DataNexus AI

Umfassendes System- & Benutzerhandbuch für entkoppelte Data Ingestion, Ontologie, RBAC-Governance, RAG-Stack & OTC Cloud Deployment

Dokumenten-Typ:	Offizielles Betriebs- & Benutzerhandbuch
Projektname:	DataNexus AI Platform
Autor & Architektur:	Thomas Nickel — AI Software Architect
Version:	v1.0 (Produktion / Enterprise Edition)
Status:	■ 100% Umgesetzt & Verifiziert (24/24 Tests grün)
Zielpattform:	Open Telekom Cloud (OTC) & Docker Compose
Datum:	29. Juli 2026

Management Summary:

Dieses Handbuch dokumentiert den vollständigen Aufbau, die Konfiguration, die Bedienung und die Wartung der **DataNexus AI** Plattform. Das System wurde mit dem Ziel entwickelt, bestehende, starre Datenimporte (z. B. historische DBLC-Pipelines) zu entkoppeln, Daten in Höchstgeschwindigkeit zu validieren und eine sichere, maschinenlesbare Schnittstelle für KI-Agenten und lokale RAG-Systeme in der Open Telekom Cloud bereitzustellen.

1. Schnellstart & Control Center Bedienung

Die Plattform bietet eine intuitive Ein-Klick-Starter-Datei `start.bat` sowie ein modernes Web Control Center (Streamlit), über das sämtliche Parameter, API-Keys, Ingestion-Importe und Tests komfortabel verwaltet werden können.

1.1 Ein-Klick-Start via `start.bat`

Im Wurzelverzeichnis des Projekts befindet sich die Starter-Datei `start.bat`. Ein Doppelklick führt automatisch folgende Schritte aus:

- **Virtuelle Umgebung:** Aktiviert das Python venv und prüft alle benötigten Packages.
- **FastAPI Backend Server:** Startet den REST-API Server im Hintergrund auf Port 8000 mit Auto-Reload.
- **Streamlit Control Center:** Startet die Web-Oberfläche auf Port 8501 und öffnet sie automatisch im Browser.

`start.bat`

REM Ausführen im Windows Explorer oder Terminal: `start.bat`

1.2 Die 5 Haupt-Bereiche des Web Control Centers (<http://localhost:8501>)

Tab / Bereich	Funktionsbeschreibung & Features
■■ Konfiguration & API-Keys	Verwaltung & Speicherung der API-Keys (Admin, Analyst, Reporting) sowie Infrastruktur-Pfade (Database, Ollama, Milvus).
■ Test-Runner & Diagnose	Ausführen aller 24 Pytest-Tests per Mausklick mit Live-Ergebnisanzeige sowie System Health Check.
■ Ingestion & Quarantäne	Drag-and-Drop CSV Upload-Interface und Live-Tabellen für importierte Datensätze und isolierte Quarantäne-Fehler.
■ RAG & KI-Playground	Indizieren neuer Dokumente, Ausführen von Vektorsuchen & Qwen LLM Inferenz sowie rollenbasierte Skill-Ausführung.
■ Betriebshandbuch & Doku	Integrierter Markdown-Viewer zum direkten Lesen aller Handbücher und Berichte im Dashboard.

2. Standalone Data Ingestion Engine (Stufe 1)

Das Ingestion-Modul arbeitet vollständig entkoppelt von Altsystemen (DBLC). Ein ereignisgesteuerter **Watchdog Service** überwacht den Eingangsordner `incoming/` (bzw. FTP-Pfade) auf eintreffende CSV-Dateien.

2.1 High-Speed Parsing & Schema Validation (Polars)

Für den Parsing-Vorgang wird die Rust-basierte Polars Dataframe Engine genutzt. Spaltennamen werden automatisch normalisiert. Fehlt eine Pflichtspalte oder liegt ein falscher Datentyp vor, greift die Schema-Validierung.

2.2 Quarantäne-Handling & Dead-Letter Queue

- **Einzelne fehlerhafte Zeilen:** Werden aus dem Import gefiltert und in der Tabelle quarantine_records mit exakter Zeilennummer und Fehlergrund isoliert. Valide Zeilen werden trotzdem importiert.
- **Komplett unlesbare Dateien:** Werden vom Quarantäne-Manager sofort in den Ordner quarantine/ verschoben. Eine Metadaten-Datei .reason.txt protokolliert die Fehlerursache.

3. Ontologie & Semantik Layer (Stufe 2)

Da Datenbanken oft kryptische Spaltenbezeichnungen wie col_49_xyz enthalten, stellt die **Ontologie Registry** ein maschinenlesbares Wörterbuch bereit. Es ordnet jeder Spalte verständliche deutsche Begriffe, Datentypen, Einheiten und PII-Sicherheitsflags zu.

Automatischer Text2SQL System-Prompt Generator:

Das System generiert aus der Ontologie dynamische System-Prompts für KI-Agenten:

```
### ENTERPRISE DATABASE SCHEMA ONTOLOGY ###
Tabelle: `ingested_records` (EnterpriseMetrics)
Spalten: - `entity_id` (customer_or_device_identifier, Typ: STRING): Eindeutige Kundennummer
[PII - Gesichert] - `metric_name` (kpi_metric_name, Typ: STRING): Name der Geschäftskennzahl
- `metric_value` (kpi_metric_value, Typ: FLOAT): Numerischer Wert der Kennzahl
```

4. Data-Access Layer & RBAC Governance (Stufe 2)

KI-Agenten kommunizieren ausschließlich über den abgesicherten **FastAPI Service**. Über den HTTP Header X-API-Key wird die Identität des Agenten geprüft und seine Rolle aufgelöst.

Rolle (RBAC)	API Key (Standard)	Zugriffsrechte & PII Maskierung
Admin	key_admin_secret_123	Vollzugriff auf alle Tabellen, Rohdaten und PII-Spalten (z. B. Kundennummern).
Analyst	key_analyst_secret_456	Zugriff auf Fachdaten und Kundennummern zur Analyse.
Reporting	key_reporting_secret_789	Eingeschränkter Zugriff. PII-Spalten werden automatisch durch die RBAC-Engine mit [RESTRICTED_BY_RBAC] anonymisiert.

5. Lokaler RAG-Stack & Qwen LLM Inferenz (Stufe 3)

Für unstrukturierte Dokumente (Verträge, PDFs, Handbücher) kommt ein lokaler RAG-Stack (Retrieval-Augmented Generation) zum Einsatz. Sämtliche Daten verbleiben zu 100% im deutschen Rechenzentrum (DSGVO-konform).

5.1 Vektor-Datenbank (Milvus) & Embeddings

- **Milvus Vector DB:** Vektordatenbank zur Speicherung von Dokumentensegmenten.
- **Embedding Engine:** Nutzung deutscher Text-Embeddings (BGE-M3) zur Durchführung von Kosinus-Ähnlichkeitssuchen.

5.2 Lokale Qwen LLM Inferenz via Ollama

Die Inferenz erfolgt über den lokalen Ollama-Dienst (<http://localhost:11434>). Das System erkennt automatisch installierte Modelle wie qwen2.5-coder:14b, qwen2.5-coder:32b oder deepseek-r1:8b.

6. Agentic Framework & Skill-Routing (Stufe 4)

Anstelle von unberechenbarem ReAct-LLM-Looping verwendet DataNexus AI ein **modulares Skill-Framework mit statischen Routen**. Jeder Agenten-Aufruf wird deterministisch gelenkt:

- `text2sql_query`: Führt strukturierte SQL-Analysen basierend auf der Ontologie aus.
- `document_rag_search`: Sucht in unstrukturierten Dokumenten und generiert Antworten via Qwen LLM.
- `data_health_check`: Überprüft System-Gesundheit, Ingestion-Quoten und Quarantäne-Raten.

7. Cloud Deployment & HITNET Security (Stufe 5)

Die Plattform wird in der **Open Telekom Cloud (OTC)** auf Elastic Cloud Servern (ECS) betrieben. Die Absicherung erfolgt über Nginx TLS 1.3 und IP-Whitelisting (HITNET-Sicherheitsstandard).

Automatisierter Cloud-Rollout:

PowerShell Skript für Windows Administration:

```
.\deploy\deploy_otc.ps1 -OtcHost "otc-ecs-instance.telekom.de" -User "ubuntu"
```

8. Verifikation & Wartung

8.1 Testergebnis der automatisierten Test-Suite (24/24 Bestanden)

Das Gesamtsystem wird durch 24 automatisierte Pytest-Unit- und Integrationstests abgedeckt. Die Testsuite garantiert höchste Zuverlässigkeit bei jeder Änderung:

Test-Modul	Anzahl Tests	Prüfbereich	Ergebnis
test_ingestion.py	3 Tests	CSV Parsing, Validation & Quarantäne	■ PASSED
test_stage2_ontology_rbac.py	5 Tests	Ontologie Registry & RBAC PII Anonymisierung	■ PASSED
test_stage3_rag_qwen.py	6 Tests	Embeddings, Vector Store & Qwen RAG Pipeline	■ PASSED
test_stage4_agentic_skills.py	6 Tests	Agent Router, Statisches Routing & Skills	■ PASSED
test_stage5_cloud_deploy ment.py	4 Tests	Nginx HITNET Config, Grafana & Scripts	■ PASSED
GESAMT	24 Tests	Vollständige Systemabdeckung	■ 100% GRÜN

HINWEIS ZUR BETRIEBSÜBERNAHME:

Durch die Kombination aus automatisierten Pytest-Tests, deklarativer Docker-Infrastruktur, importierbarem Grafana Dashboard as Code und diesem umfassenden Betriebshandbuch ist das System **vollkommen unabhängig von einzelnen Entwicklern betreibbar** (Abbau der Key-Person-Dependency).